13. AVS 컨트랙트 및 바인딩 분석

개요

Omni AVS (Actively Validated Service)는 EigenLayer 프로토콜과 통합되어 Ethereum의 검증자들이 Omni 네트워크의 보안에 참여할 수 있도록 하는 시스템입니다. 이 문서는 AVS 컨트랙트의 구조와 Go 바인딩 생성 과정을 상세히 분석합니다.

1. OmniAVS 컨트랙트 구조

1.1 핵심 상속 구조

```
contract OmniAVS is
IOmniAVS,
IOmniAVSAdmin,
OwnableUpgradeable,
PausableUpgradeable,
OmniAVSStorage
```

주요 특징:

- **EigenLayer 통합**: DelegationManager와 AVSDirectory 연동
- **업그레이드 가능**: OpenZeppelin의 업그레이드 가능한 컨트랙트 패턴
- **일시 정지 기능**: 보안 상황에서 컨트랙트 기능 중단 가능
- **소유권 관리**: 관리자 기능과 일반 사용자 기능 분리

1.2 생성자 및 초기화

생성자:

```
constructor(IDelegationManager delegationManager_, IAVSDirectory avsDirectory_)

_delegationManager = delegationManager_;

_avsDirectory = avsDirectory_;

_disableInitializers();

}
```

초기화 파라미터:

```
function initialize(

address owner_, // 컨트랙트 소유자

IOmniPortal omni_, // Omni 포털 컨트랙트

uint64 omniChainId_, // Omni 체인 ID

address ethStakeInbox_, // EthStakeInbox 주소

uint96 minOperatorStake_, // 최소 오퍼레이터 스테이크

uint32 maxOperatorCount_, // 최대 오퍼레이터 수

StrategyParam[] calldata strategyParams_, // 전략 파라미터

string calldata metadataURI_, // 메타데이터 URI

bool allowlistEnabled_ // 허용 목록 활성화 여부

) external initializer
```

2. 오퍼레이터 등록 시스템

2.1 등록 과정

registerOperator 함수:

```
function registerOperator(
bytes calldata pubkey,
ISignatureUtils.SignatureWithSaltAndExpiry memory operatorSignature
) external whenNotPaused
```

등록 검증 단계:

1. **공개키 검증:**

```
require(operator == Secp256k1.pubkeyToAddress(pubkey), "OmniAVS: pubkey != sende
```

2. **허용 목록 확인:**

```
require(!allowlistEnabled || _allowlist[operator], "OmniAVS: not allowed");
```

3. **중복 등록 방지:**

```
require(!_isOperator(operator), "OmniAVS: already an operator");
```

4. **최대 오퍼레이터 수 확인:**

```
require(_operators.length < maxOperatorCount, "OmniAVS: max operators reached");
```

5. **최소 스테이크 요구사항:**

```
require(_getTotalDelegations(operator) >= minOperatorStake, "OmniAVS: min stake
```

2.2 공개키 검증 라이브러리

Secp256k1 라이브러리:

```
library Secp256k1 {
function pubkeyToAddress(bytes memory pubkey) internal pure returns (address) {
require(pubkey.length == 64, "Secp256k1: invalid pubkey length");
return address(uint160(uint256(keccak256(pubkey))));
}
}
```

특징:

- 64바이트 압축되지 않은 secp256k1 공개키 사용
- 0x04 접두사 없이 순수 좌표값만 사용
- Keccak256 해시를 통해 Ethereum 주소 생성

3. 스테이킹 및 위임 시스템

3.1 전략 파라미터

StrategyParam 구조:

```
struct StrategyParam {
IStrategy strategy; // EigenLayer 전략 컨트랙트
uint96 multiplier; // 스테이크 가중치 승수
}
```

3.2 스테이크 계산

총 위임량 계산:

```
function _getTotalDelegations(address operator) internal view returns (uint96) {
uint96 total;
StrategyParam memory params;

for (uint256 i = 0; i < _strategyParams.length;) {
params = _strategyParams[i];
uint256 shares = _delegationManager.operatorShares(operator, params.strategy);
total += _weight(shares, params.multiplier);
unchecked { i++; }
}
return total;
}
```

가중치 계산:

```
function _weight(uint256 shares, uint96 multiplier) internal pure returns (uint9

return uint96(shares * multiplier / STRATEGY_WEIGHTING_DIVISOR);

}
```

3.3 오퍼레이터 정보 구조

Operator 구조체:

```
struct Operator {

address addr; // 오퍼레이터 주소

bytes pubkey; // 64바이트 공개키

uint96 delegated; // 위임받은 스테이크 (자체 스테이크 제외)

uint96 staked; // 자체 스테이크

}
```

4. Omni 체인과의 동기화

4.1 syncWithOmni 함수

```
function syncWithOmni() external payable whenNotPaused {

Operator[] memory ops = _getOperators();

omni.xcall{ value: msg.value }(

omniChainId,

ConfLevel.Finalized,
```

```
ethStakeInbox,
abi.encodeWithSelector(IEthStakeInbox.sync.selector, ops),
_xcallGasLimitFor(ops.length)
);
}
```

동기화 과정:

1. 현재 등록된 모든 오퍼레이터 정보 수집
2. OmniPortal을 통해 크로스체인 호출 실행
3. Omni 체인의 EthStakeInbox 컨트랙트로 데이터 전송
4. 최종화된 확인 레벨(ConfLevel.Finalized) 사용

4.2 가스 한도 계산

```
function _xcallGasLimitFor(uint256 numOperators) internal view returns (uint64)
return uint64(numOperators) * xcallGasLimitPerOperator + xcallBaseGasLimit;
}
```

수수료 계산:

```
function feeForSync() external view returns (uint256) {
Operator[] memory ops = _getOperators();
return omni.feeFor(
omniChainId,
```

```
abi.encodeWithSelector(IEthStakeInbox.sync.selector, ops),
_xcallGasLimitFor(ops.length)
);
}
```

5. 허용 목록 시스템

5.1 허용 목록 관리

허용 목록 활성화:

```
function _enableAllowlist() private {
require(!allowlistEnabled, "OmniAVS: already enabled");
allowlistEnabled = true;
emit AllowlistEnabled();
}
```

오퍼레이터 추가:

```
function addToAllowlist(address operator) external onlyOwner {
require(operator != address(0), "OmniAVS: zero address");
require(!_allowlist[operator], "OmniAVS: already in allowlist");
_allowlist[operator] = true;
emit OperatorAllowed(operator);
```

```
}
```

5.2 등록 가능성 검증

```
function canRegister(address operator) external view returns (bool, string memor

if (!_delegationManager.isOperator(operator))

return (false, "not an operator");

if (allowlistEnabled && !_allowlist[operator])

return (false, "not in allowlist");

if (_isOperator(operator))

return (false, "already registered");

if (_operators.length >= maxOperatorCount)

return (false, "max operators reached");

if (_getTotalDelegations(operator) < minOperatorStake)

return (false, "min stake not met");

return (true, "");

}
```

6. 스토리지 레이아웃

6.1 OmniAVSStorage 컨트랙트

```
abstract contract OmniAVSStorage {
```

```
IOmniAVS.StrategyParam[] internal _strategyParams; // 전략 파라미터 배열

address[] internal _operators; // 등록된 오퍼레이터 주소 배열

mapping(address => bytes) internal _operatorPubkeys; // 오퍼레이터 공개키 매핑

mapping(address => bool) internal _allowlist; // 허용 목록 매핑

uint32 public maxOperatorCount; // 최대 오퍼레이터 수

uint64 public omniChainId; // Omni 체인 ID

uint64 public xcallGasLimitPerOperator; // 오퍼레이터당 가스 한도

uint64 public xcallBaseGasLimit; // 기본 가스 한도

uint96 public minOperatorStake; // 최소 오퍼레이터 스테이크

bool public allowlistEnabled; // 허용 목록 활성화 여부

address public ethStakeInbox; // EthStakeInbox 주소

IOmniPortal public omni; // Omni 포털 컨트랙트

}
```

7. Go 바인딩 생성 시스템

7.1 바인딩 생성 스크립트

gen.sh 스크립트:

```
gen_binding() {

contract=$1

# 컨트랙트 이름 추출

name=$(echo ${contract} | cut  -d  ":"  -f  2)
```

```
name_lower=$(echo ${name} | tr '[:upper:]' '[:lower:]')
# ABI 및 바이트코드 추출
temp=$(mktemp -d)
forge inspect --root ${ROOT} ${contract} abi --json > ${temp}/${name}.abi
forge inspect --root ${ROOT} ${contract} bytecode > ${temp}/${name}.bin
# Go 바인딩 생성
abigen \
--abi ${temp}/${name}.abi \
--bin ${temp}/${name}.bin \
--type ${name} \
--pkg bindings \
--out ./bindings/${name_lower}.go
}
```

7.2 생성된 Go 바인딩 구조

OmniAVS 바인딩 예시:

```
// 자동 생성된 구조체
type IOmniAVSOperator struct {
Addr common.Address
Pubkey []byte
Delegated *big.Int
Staked *big.Int
```

```
}


type IOmniAVSStrategyParam struct {
Strategy common.Address
Multiplier *big.Int
}


// 메타데이터
var  OmniAVSMetaData = &bind.MetaData{
ABI: "[{\"type\":\"constructor\",\"inputs\":[...]}]",
Bin: "0x60c06040523480156200001157600080fd5b50...",
}
```

7.3 바인딩 빌드 프로세스

Makefile 타겟:

```
AVS_CONTRACTS := OmniAVS DelegationManager StrategyManager StrategyBase AVSDirec


bindings: check-abigen-version build
./bindings/scripts/enumgen.sh
ROOT=./avs ./bindings/scripts/gen.sh $(AVS_CONTRACTS)
# 타입 주석 처리
```

```
go run ./bindings/scripts/commenttypes.go -- bindings/avsdirectory.go ISignature
```

도구 버전 관리:

- **abigen**: v1.14.8 (go-ethereum)
- **forge**: Foundry 도구체인
- **pnpm**: v9.12.1 (의존성 관리)

8. EigenLayer 통합

8.1 핵심 EigenLayer 컴포넌트

DelegationManager:

- 오퍼레이터 위임 관리
- 스테이크 계산 및 추적
- 위임자-오퍼레이터 관계 관리

AVSDirectory:

- AVS 등록 및 메타데이터 관리
- 오퍼레이터-AVS 관계 추적
- 서명 검증 및 등록 처리

StrategyManager:

- 다양한 스테이킹 전략 관리
- 토큰별 스테이킹 로직
- 출금 및 슬래싱 처리

8.2 AVS 호환성

IServiceManager 인터페이스 구현:

```
function avsDirectory() external view returns (address) {
return address(_avsDirectory);
}

function getRestakeableStrategies() external view returns (address[] memory) {
return _getRestakeableStrategies();
}

function getOperatorRestakedStrategies(address operator)
external view returns (address[] memory) {
if (!_isOperator(operator)) return new address[](0);
return _getRestakeableStrategies();
}
```

9. 보안 및 거버넌스

9.1 접근 제어

소유자 전용 함수:

- `setMetadataURI` : AVS 메타데이터 업데이트
- `setStrategyParams` : 스테이킹 전략 설정
- `addToAllowlist` / `removeFromAllowlist` : 허용 목록 관리

- `ejectOperator` : 강제 오퍼레이터 제거

일시 정지 기능:

- `pause` / `unpause` : 전체 컨트랙트 일시 정지
- `whenNotPaused` 모디파이어로 핵심 함수 보호

9.2 이벤트 시스템

주요 이벤트:

```
event OperatorAdded(address indexed operator);

event OperatorRemoved(address indexed operator);

event OperatorAllowed(address indexed operator);

event OperatorDisallowed(address indexed operator);

event AllowlistEnabled();

event AllowlistDisabled();

event StrategyParamsSet(StrategyParam[] params);
```

10. 사용 사례 및 워크플로우

10.1 오퍼레이터 등록 워크플로우

1. **EigenLayer에서 오퍼레이터 등록**
2. **충분한 스테이크 확보** (minOperatorStake 이상)
3. **허용 목록 추가** (활성화된 경우)
4. **공개키와 서명으로 AVS 등록**
5. **정기적인 Omni 체인 동기화**

10.2 스테이크 관리 워크플로우

1. **전략 파라미터 설정** (관리자)
2. **오퍼레이터 스테이킹** (EigenLayer)
3. **위임자 위임** (EigenLayer)
4. **AVS 동기화 호출** (누구나)
5. **Omni 체인 상태 업데이트**

결론

Omni AVS는 EigenLayer의 강력한 스테이킹 인프라를 활용하여 Omni 네트워크의 보안을 강화하는 핵심 컴포넌트입니다. 체계적인 오퍼레이터 관리, 유연한 스테이킹 전략, 그리고 자동화된 Go 바인딩 생성 시스템을 통해 개발자 친화적이면서도 안전한 크로스체인 검증 시스템을 제공합니다. 특히 허용 목록 시스템과 일시 정지 기능을 통해 점진적인 네트워크 확장과 보안 관리가 가능하도록 설계되었습니다.